Learning Parametric ADCIRC-Like Coastal Shallow-Water Dynamics with Physics-Informed and Operator-Learning Surrogate Models

Roberto Hogas Goras

***Abstract*—High-fidelity hydrodynamic models such as the Advanced CIRCulation model (ADCIRC) are widely used for coastal-flow and storm-surge simulations. Furthermore, many data-driven coastal prediction approaches rely heavily on large collections of numerical simulations or extensive historical coastal observations. In this paper, we investigate whether simplified ADCIRC-like shallow-water dynamics can be directly approximated from a small independently generated synthetic ensemble of scenarios using two different deep-learning-based surrogate approaches: a Physics-Informed Neural Network (PINN) and a Deep Operator Network (DeepONet). In this way, we explore whether AI-based surrogate models can be developed with the goal of reducing direct dependency on extensive historical reanalysis products such as NOAA's Coastal Ocean Reanalysis (CORA) dataset. A numerical solver for the one-dimensional Saint-Venant equations is used to generate independent scenarios with randomized bathymetric and physical configurations, varying the tidal forcing and wind speed, among other parameters. The PINN and DeepONet surrogate models are then trained to predict the spatio-temporal free-surface elevation and depth-averaged velocity state variables. Their performance is compared in terms of predictive accuracy, generalization to an independently generated unseen scenario, computational cost, and satisfaction of the governing equations. DeepONet achieves lower pointwise errors in the generalization assessment, whereas the PINN shows greater physical consistency with the governing PDEs, as reflected by lower continuity and momentum residuals. These results highlight contrasting trade-offs between the two deep-learning surrogate models and provide a preliminary step toward the development and implementation of a comprehensive AI-based surrogate framework for more complex ADCIRC-type coastal simulations.**

***Index Terms*—Physics-informed neural networks, PINN, DeepONet, neural operators, shallow-water equations, parametric PDEs, storm surge modeling, surrogate modeling, coastal hydrodynamics, ADCIRC**

I. Introduction

OVER the past several decades, tropical cyclones have resulted in enormous societal and economic costs in the United States. In the United States alone, the associated damages due to these storm systems between 1980 and 2024 are estimated to exceed $34 billion annually (NOAA National Centers for Environmental Information, 2025). For instance, the hurricane Otis, which caused caused approximately at least 52 deaths and $16 billion in damage, was the strongest storm ever recorded at landfall in the eastern Pacific, striking with catastrophic winds of 143 knots in October (Reinhart and Reinhart, 2024). Evidence since the 1970s points to an apparent upward trend in tropical cyclone intensity and destructive potential (Emanuel, 2005). Another natural disaster that has caused widespread devastation is the tsunami. For instance, more than 200,000 people died in 14 countries around the Indian Ocean as a result of the 2004 Sumatran-Andaman tsunami (Lay et al., 2005), and there were also significant financial losses. These events are not only highly destructive, but also important drivers of geological change, reshaping coastal landforms, sedimentary systems, and habitats. Approximately 60% of the global population lives in coastal regions, increasing the exposure of communities to coastal hazards (Hinrichsen, 1998). For that reason, accurate storm-surge prediction is essential to assess the long-term risks that tropical cyclones and hurricanes pose to coastal infrastructure and population. Numerous studies have emerged over the last decades focusing on the modeling and prediction of storm surge. These methods range from probabilistic and ensemble frameworks that quantify uncertainty in surge forecasts (Bernier and Thompson, 2015) to deterministic models that simulate specific storm-surge events (Lin et al., 2010).

A widely used numerical model is the Advanced Multi-Dimensional CIRCulation Model for Shelves, Coasts and Estuaries (ADCIRC), developed by Luettich et al. (1992) and Westerink et al. (1994), a hydrodynamic model that addresses the shallow-water equations using the continuous Galerkin linear finite element approach on unstructured grids (Westerink et al., 1992). It serves as the primary physics-based modeling component in several forecasting systems like the Coastal Emergency Risk Assessment (CERA) framework. Many papers have demonstrated that ADCIRC has become one of the most established physics-based tools for storm-surge prediction and coastal modeling, with applications ranging from historical hurricane reconstruction to real-time coastal flood guidance. For instance, this numerical model has succesfully simulated compound flooding during Hurricane Harvey (2017), with rainfall-induced runoff incorporated into the model through riverine boundary conditions (Loveland et al., 2021; Pachev et al., 2023b).

Although ADCIRC has historically been used for storm surge modeling, driven primarily by extreme winds from hurricanes and tropical cyclones, tsunamis generally have long wavelengths and exhibit limited dispersion. Therefore, their propagation and coastal inundation can also be modeled using shallow-water equations (Wang et al., 2021a; Kolukula et al., 2025). With the use of ADCIRC, simulated tsunami propagation accurately and efficiently over a global computational domain. Beyond direct hazard forecasting, ADCIRC has also been used with Lagrangian particle-tracking models to

assess environmental risk from coastal infrastructure decisions, such as simulating oil spill transport near proposed crude oil loading terminals under both normal and hurricane-driven flow conditions (Reich et al., 2025).

Since ADCIRC can be very expensive in terms of computational cost due to its infeasiblity of managing large input datasets, The use of deep learning and machine learning surrogate models for forecasting these temporal phenomena has increased exponentially in recent years. Previous studies have used a variety of machine-learning, ranging from convolutional neural networks to short-term memory (LSTM) networks and multilayer perceptrons (MLPs) (Wang and Tian, 2022; Giaremis et al., 2024; Sun et al., 2022). In Pachev et al. (2023a) a multi-stage approach is used to reduce the number of model parameters. Hou et al. (2025) applied the ADCIRC+SWAN model to encode the input variables into latent representations, which are then combined and processed to extract the most relevant features using a uniform spatiotemporal deep learning model.

However, some deep learning-based models, such as ANNs, also have drawbacks. One of these is their reliance on long-term historical weather data for training (Tiggeloven et al., 2021), typically obtained from global reanalysis datasets such as NOAA-CORA or MERRA-2. This raises an important question: how well can these models perform when extensive historical training data is unavailable or severely limited? The aim of this study is to show that we can train AI models to mimic simplified, one-dimensional shallow-water dynamics (ADCIRC-like) without relying solely on historical data.

Shallow-water equations are widely used to model and simulate different geophysical phenomena such as snow avalanches or tsunamis. As we have mentioned before, they are also commonly used to represent surface-water dynamics across a wide variety of spatial scales, ranging from local channels and coastal regions to large-scale oceanic domains. More complex applications involve flow simulations featuring a high number of interacting waves, extreme forcing conditions, variable boundary conditions, and complex open-boundary bathymetries to be modeled (Westerink et al., 2008; Dietrich et al., 2011; Hope et al., 2013; Yamazaki et al., 2011). They are derived by depth-integrating the incompressible Navier-Stokes equations, assuming that the horizontal length scales greatly exceeds the vertical scales. As a consequence, the shallow-water equations use a single variable simplification of the underlying vertical velocity profile, also described as the depth-averaged velocity. However, in Kern et al. (2010) and Sanvitale and Bowman (2016) There is talk that this simplification may be excessive, which could lead to inconsistencies and inaccuracies.

II. Governing One-Dimensional Shallow-Water Equations

Following the formulation presented by Morales-Hernández et al. (2013), the one-dimensional Saint–Venant, or shallow-water, equations can be written as the following system of conservation laws:

$$\frac{\partial U(x,t)}{\partial t}+\frac{\partial F(x,U)}{\partial x}=H(x,U), \tag{1}$$

$$U=\begin{pmatrix}A\\Q\end{pmatrix}, \tag{2}$$

$$F=\begin{pmatrix}Q\\\dfrac{Q^2}{A}+gI_1\end{pmatrix}, \tag{3}$$

$$H=\begin{pmatrix}0\\g\left[I_2+A(S_0-S_f)\right]\end{pmatrix} \tag{4}$$

It is a hyperbolic system of partial differential equations that represents the conservation of mass and momentum in one-dimensional free-surface flows. Regarding the vectors, U is the state vector, which contains the conserved quantities A and Q, whose evolution over time is being solved. Por un lado, A representa el wetted-cross sectional area of the channel. To define it, some parameters must first be introdueced. The absolute free-surface elevation at any spatiotemporal point (x,t) is defined as

$$z_s(x,t)=H_0+\zeta(x,t), \tag{5}$$

where H_0 is the reference still-water level and $\zeta(x,t)$ is the free-surface displacement relative to that level. Consequently, the total water depth is

$$h(x,t)=z_s(x,t)-z_b(x)=H_0+\zeta(x,t)-z_b(x). \tag{6}$$

The wetted cross-sectional area is then calculated as

$$A(x,t)=\int_{z_b(x)}^{z_s(x,t)}\sigma(x,\eta)\,\mathrm{d}\eta, \tag{7}$$

where $\sigma(x,\eta)$ is the channel width at spatial position x and elevation η. On the other hand, Q is the volumetric water discharge through the cross section A, y se expresa como:

$$Q(x,t)=A(x,t)u(x,t), \tag{8}$$

where $u(x,t)$ is the cross-sectionally averaged velocity. Then, F is the flux vector, describing the transport of mass and momentum along the channel, while H is the source-term vector. The latter represents the external and geometric contributions to the momentum balance. Furthermore, g denotes the acceleration due to gravity.

Additionally, S_f represents the friction slope associated with momentum loss, which is caused by the resistance exerted by the channel bed and walls on the water flow. It is modeled using Manning's empirical formulation.

$$S_f=\frac{Q^2n^2}{A^2R^{4/3}}, \tag{9}$$

where R denotes the hydraulic radius and n is Manning's roughness coefficient.

S_0 is the bed slope, that is, the measure that represents how does z_b change along the channel. It is defined as follows:

$$S_0=-\frac{\partial z_b}{\partial x}, \tag{10}$$

where $z_b(x)$ denotes the bed elevation evaluated at each spatial grid point x of the discretized channel domain.

Regarding the forces, I_1 represents the hydrostatic pressure at the cross-section A with water depth h. It can be calculated using the following integral:

$$I_1(x) = \int_0^h (h - \eta)\, \sigma(x, \eta)\, d\eta \tag{11}$$

As it can be deduced, x is the horizontal coordinate measured along the flow, and η is the vertical coordinate measured upwards from the channel bottom within the domain. Recall that $\sigma(x, \eta)$ is the width of the cross-section at height η.

In contrast, I_2 is the pressure force due to changes in channel width along the flow direction. It is defined by the following integral:

$$I_2(x) = \int_0^h (h - \eta)\, \frac{\partial \sigma(x, \eta)}{\partial x}\, d\eta \tag{12}$$

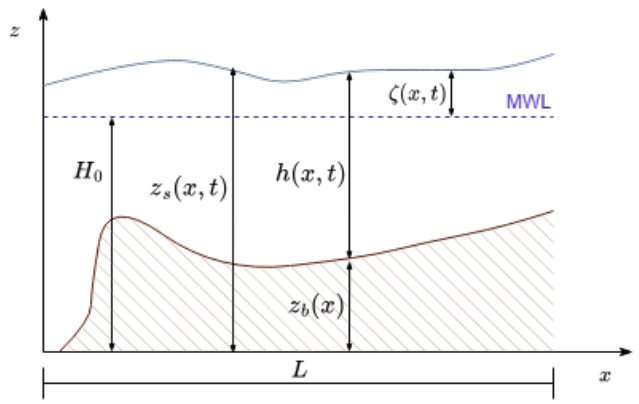


Fig. 1: Schematic longitudinal view of the 1D shallow-water configuration, showing the bed elevation $z_b(x)$, the free-surface level $z_s(x,t)$, the surface elevation relative to the Mean Water Level (MWL) $\zeta(x,t)$, the water depth h(x,t), the reference still-water depth H_0, and the domain length L.

There are other ways to express the Saint-Venant equations in one spatial dimension. In addition to the conservative form, they can also be expressed in non-conservative form, which is used in contexts such as the method of characteristics (MOC) to simulate dam-break type flows (Clain and Figueiredo, 2017). To derive this form, the chain rule must be applied to the flux vector F, separating its explicit dependence on the state vector U:

$$\frac{d\mathbf{F}(x, \mathbf{u})}{dx} = \frac{\partial F(x, u)}{\partial x} + J(x, u)\frac{\partial u}{\partial x} \tag{13}$$

where $\mathbf{J} = \frac{\partial \mathbf{F}}{\partial \mathbf{u}}$ is the Jacobian matrix of the original system. Rewriting the H vector, the source term, in the following way

$$\tilde{H}(x, u) = H(x, u) - \frac{\partial F(x, u)}{\partial x}, \tag{14}$$

we can obtain the non-conservative form

$$\frac{\partial u(x, t)}{\partial t} + J(x, u)\frac{\partial u(x, t)}{\partial x} = \tilde{H}(x, u). \tag{15}$$

However, for this work, we have focused on the conservative form, since the non-conservative form, despite having certain applications mentioned earlier, presents significant disadvantages in numerical analysis (Kurganov et al., 2025). This is primarily because it does not guarantee the conservation of mass or linear momentum when discretizing a specific domain, thereby affecting the accurate representation of wave propagation, tidal, water-level variations, etc. (Hodges, 2019). It also introduces a great numerical dissipation.

III. PINN

First introduced by Dissanayake and Phan-Thien (1994), neural networks (NNs) have been used in many PDE-driven scenarios to approximate the solutions to such equations, leveraging the universal approximation property (Cybenko, 1989). Since this milestone, numerous strategies have been developed to exploit the predictive and optimization capabilities of NNs. Physics-Informed Neural Networks (PINNs) are a type of deep learning model introduced by (Raissi et al., 2019) that aim to train an NN using governing nonlinear PDEs by minimizing an optimization loss function that measures the violation of the equations themselves. This approach differs from classical deep learning models based on neural networks, which are trained exclusively on an available training dataset; this can lead to "greedy" behavior that triggers the well-known phenomenon of overfitting. In other words, let $\mathcal{D}_{\text{train}} = \{(\mathbf{x}_i, \mathbf{y}_i)\}_{i=1}^N \subset \mathcal{X} \times \mathcal{Y}$ denote the training dataset. A conventional data-driven NN is trained by solving the optimization objective function

$$\boldsymbol{\theta}^* = \arg\min_{\boldsymbol{\theta}} \mathcal{L}_{\text{data}}(\boldsymbol{\theta}), \tag{16}$$

where

$$\mathcal{L}_{\text{data}}(\boldsymbol{\theta}) = \frac{1}{N}\sum_{i=1}^{N} \|\hat{\mathbf{y}}_{\boldsymbol{\theta}}(\mathbf{x}_i) - \mathbf{y}_i\|_2^2 \tag{17}$$

If the value of $\boldsymbol{\theta}^*$ is small after training, it indicates that the neural network fits the available training observations well but cannot guarantee accurate predictions on unseen data points. Overfitting arises when the neural network is over-parameterized and trained on a limited dataset, causing the model to struggle with generalization. Consequently, the absolute error between its predictions and the actual values of unseen points can be significantly higher than the error obtained on the training data.

PINN, on the other hand, can avoid, at least partially, these overfitting issues. It addresses this limitation by incorporating the PDE governing equations and the associated initial and boundary condition losses into the training objective function. The total of each loss component is not computed using all spatiotemporal points of the parameterized domain, but only at selected points called collocation points. Optionally, a supervised data loss may also be incorporated when labeled reference solutions are available. When the physical parameters are introduced into the network inputs and the PINN model is trained on a set of multiple PDE scenarios, the resulting PINN can be used as a parametric surrogate model, as we have approached in this work. However, it is worth mentioning that the incorporation of these physical-based constraints does not guarantee complete generalization nor

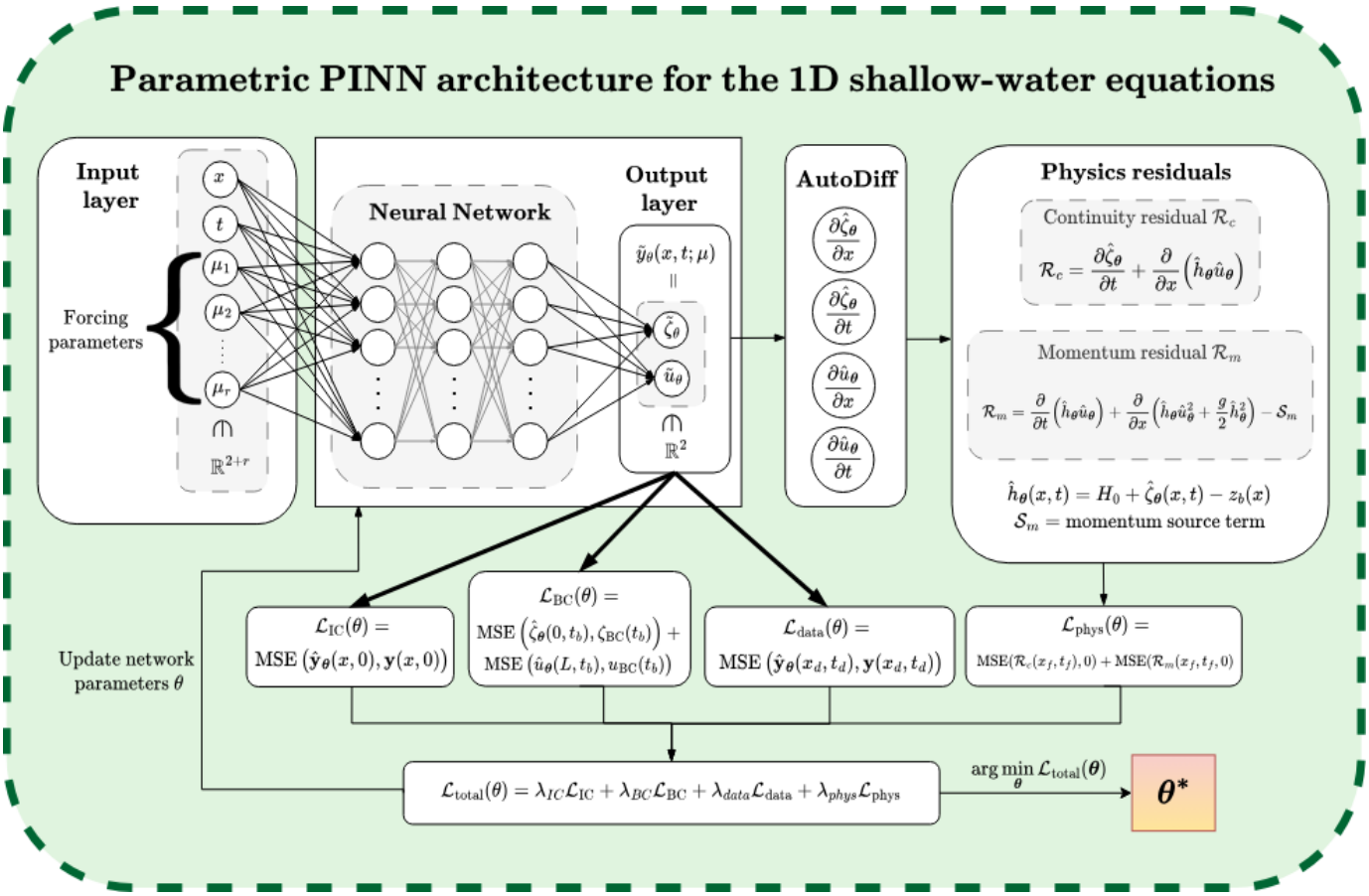


Fig. 2: Schematic showing the PINN architecture and training workflow used for the surrogate modeling of the parametric one-dimensional shallow-water equations considered in this work. A single feedforward neural network is employed, whose inputs are the spatio-temporal coordinaes (x, t) and a physical-parameter vector denoted by $\boldsymbol{\mu} = (\mu_1, \ldots, \mu_r)$. This vector may include storm properties, wind conditions, etc. (conditions that change between training samples).

does it completely eliminate overfitting. Following common deep learning terminology, for each loss component, let

$$\left\{\boldsymbol{\chi}_i^{(k)}\right\}_{i=1}^{N_k}, \qquad k \in \{\text{data, phys, IC, BC}\}, \tag{18}$$

denote the corresponding set of sampled input points, where

$$\boldsymbol{\chi}_i^{(k)} = \begin{pmatrix} x_i^{(k)} \\ t_i^{(k)} \\ \boldsymbol{\mu}_i^{(k)} \end{pmatrix} \in \mathbb{R}^{2+r} \tag{19}$$

Here, N_k is the number of points associated with loss component k, whereas r is the number of physical parameters collected in the vector $\boldsymbol{\mu} = (\mu_1, \mu_2, \ldots, \mu_r)^{\mathsf{T}} \in \mathbb{R}^r$.

The PINN model, represented as $\text{NN}_{\boldsymbol{\theta}}$ defines the mapping

$$\text{NN}_{\boldsymbol{\theta}} : \mathbb{R}^{2+r} \longrightarrow \mathbb{R}^q, \qquad \boldsymbol{\chi}_i \longmapsto \hat{\mathbf{y}}_{\boldsymbol{\theta}}(\boldsymbol{\chi}_i), \tag{20}$$

where q is the number of predicted dependent variables. The optimization training problem can then be formulated as

$$\boldsymbol{\theta}^* = \arg\min_{\boldsymbol{\theta}} \mathcal{L}_{\text{total}}(\boldsymbol{\theta}), \tag{21}$$

where the total objective function is defined as

$$\begin{aligned} \mathcal{L}_{\text{total}}(\boldsymbol{\theta}) = \lambda_{\text{data}}\mathcal{L}_{\text{data}}(\boldsymbol{\theta}) + \lambda_{\text{phys}}\mathcal{L}_{\text{phys}}(\boldsymbol{\theta}) \\ + \lambda_{\text{IC}}\mathcal{L}_{\text{IC}}(\boldsymbol{\theta}) + \lambda_{\text{BC}}\mathcal{L}_{\text{BC}}(\boldsymbol{\theta}), \end{aligned} \tag{22}$$

and λ_{data}, λ_{phys}, λ_{IC} and λ_{BC} are non-negative weighting coefficients. Recall that data loss part is optional depending on whether reference data solutions are given.

Several extensions of the original PINN have been proposed in recent years to improve its stability, accuracy, scalability, and so on. For example, conservative PINNs (cPINNs) (Jagtap et al., 2020) enforce continuity and flux across the interfaces of non-overlapping subdomains. In contrast, variational PINNs (VPINNs) (Kharazmi et al., 2019) formulate the PDE residual in a variational form to lower the order of the differential operators involved in the NNs. Yang et al. (2021) introduced Bayesian PINNs (B-PINNs), which employ a probabilistic Bayesian network to quantify uncertainty arising from noise in the training dataset. Another extension is the hp-variational PINN (hp-VPINN), introduced by Kharazmi et al. (2021); it also utilizes a variational formulation but involves domain decomposition and polynomial refinement to project the PDE residual onto a space of high-order piecewise polynomials.

IV. DeepONet

The Deep Operation Network (DeepONet) is a deep learning architecture first introduced by Lu et al. (2021), based on the universal approximation theorem for operators by Chen

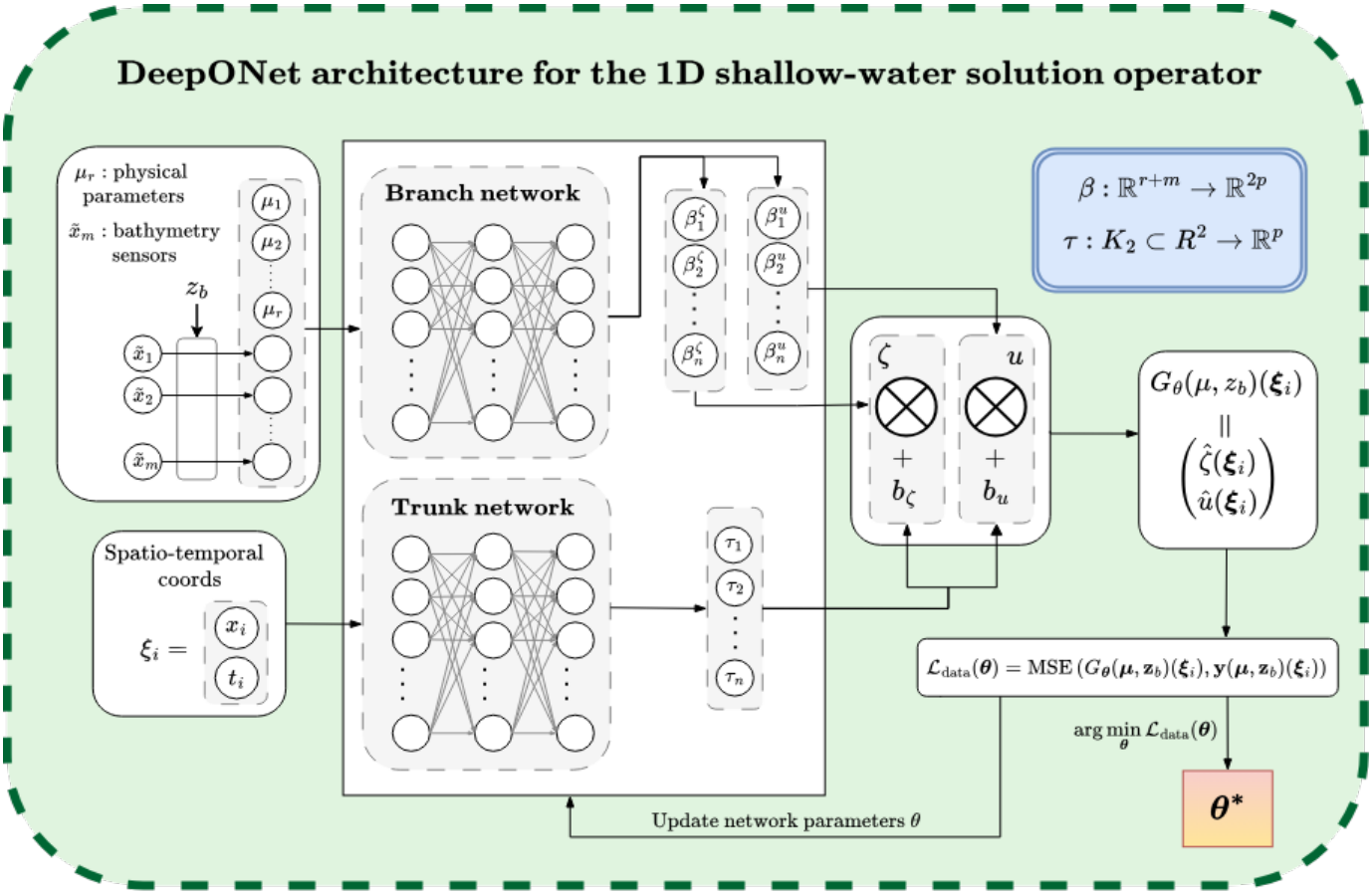


Fig. 3: DeepONet's architecture and training workflow used for the surrogate modeling of the one-dimensional shallow-water equations used in this work. As explained before, this architecture consists of two neural networks, referred to as the branch and trunk networks. After setting up an initial configuration of the PDE, the branch network receives a finite-dimensional description of it, consisting of a vector of physical parameters $\boldsymbol{\mu} = (\mu_1, \ldots, \mu_r)$ and the bathymetric profile z_b evaluated at m fixed sensor locations $\{\tilde{x}_1, \ldots, \tilde{x}_m\}$. Note that the number of physical parameters can change depending on the problem setting. One may include wind speed, tidal amplitude and frequency, storm forcing parameters like wind direction, bottom-drag coefficients, etc.

and Chen (1995). Operator learning seeks to represent mappings between infinite-dimensional function spaces defined on a bounded domain. Specifically, let $\mathcal{X}$ and $\mathcal{Y}$ be two Banach spaces of continuous functions defined on two compact (closed and bounded) domains $K_1 \subset \mathbb{R}^{d_x}$ y $K_2 \subset \mathbb{R}^{d_y}$ respectively, i.e., $\mathcal{X} = C(K_1)$ and $\mathcal{Y} = C(K_2)$. It is considered a nonlinear continuous operator

$$\mathcal{G} : \mathcal{U} \to \mathcal{S}, \tag{23}$$

where $\mathcal{U} \subseteq \mathcal{X}$ and $\mathcal{S} \subseteq \mathcal{Y}$ represent the sets of input and output functions under consideration. The goal is to approximate the underlying operator $\mathcal{G}$ constructing a parameterized model $\mathcal{G}_{\boldsymbol{\theta}}$ that follows the following mapping:

$$\mathcal{G}_{\boldsymbol{\theta}} : \mathcal{U} \to \mathcal{S}, \quad \boldsymbol{\theta} \in \boldsymbol{\Theta}, \tag{24}$$

where $\boldsymbol{\Theta} \subseteq \mathbb{R}^{N_{\boldsymbol{\theta}}}$ is a finite-dimensional parameter space and $N_{\boldsymbol{\theta}}$ denotes the total number of trainable parameters.

This architecture allows, for example, the transformation of initial conditions or other input functions directly into the corresponding spatio-temporal solution of a PDE. Such types of PDEs are also known as "parametric", in which one or more parameters of the specific PDE are allowed to vary within a specified range.

In the general DeepOnet setting, a division is made into two separate neural networks: a **branch** network, which encodes the discretized input function, and a **trunk** network, which represents the spatiotemporal coordinates of the output function. In this way, the branch network captures the dependence of the solution on the input function, while the trunk network represents its dependence on the coordinates of the output domain. Using a similar notation as in Lu et al. (2021), in the present one-dimensional shallow-water setting, these coordinates are given by $\xi_i = (x_i, t_i) \in K_2 \subset \mathbb{R}^2$, $i = 1, \ldots, N_{xt}$, where $N_{xt} = n_x \times n_t$ is the total number of space-time points. The branch network in a default setting $\boldsymbol{\beta}$ receives point-wise evaluations of the input function v at a finite set of sensor locations $\{\tilde{x}_1, \tilde{x}_2, \ldots, \tilde{x}_m\} \subset K_1$, where m is the fixed number of sensors. These evaluations defines a finite-dimensional representation of the input function $\gamma(v) := [v(\tilde{x}_1), v(\tilde{x}_2), \ldots, v(\tilde{x}_m)] \in \mathbb{R}^m$. Then it maps this encoding into a p-dimensional feature embedding $\boldsymbol{\beta}(\gamma(v)) = [\beta_1(\gamma(v)), \beta_2(\gamma(v)), \ldots, \beta_p(\gamma(v))] \in \mathbb{R}^p$, resulting in the following mapping: $\boldsymbol{\beta} : \mathbb{R}^m \to \mathbb{R}^p, \quad \gamma(u) \mapsto \boldsymbol{\beta}(\gamma(v))$.

On the other hand, the trunk network takes the coordinates $\boldsymbol{\xi}_i \subset K_2$ at which the output function is evaluated and produces another latent representation $\boldsymbol{\tau}(\xi_i) =$

$[\tau_1(\xi_i), \tau_2(\xi_i), \ldots, \tau_n(\xi_i)] \in \mathbb{R}^n$, defining the mapping $\boldsymbol{\tau} : K_2 \to \mathbb{R}^p, \quad \xi_i \mapsto \boldsymbol{\tau}(\xi_i)$.

The output of DeepONet is then obtained by taking the inner product (denoted by $\cdot$) of the branch and trunk representations:

$$G_{\boldsymbol{\theta}}(u)(\boldsymbol{\xi}_i) = \sum_{k=1}^{q} \beta_k(v) \cdot \tau_k(\boldsymbol{\xi}_i), \quad v \in \mathcal{U}, \quad \boldsymbol{\xi}_i \in K_2 \tag{25}$$

Although it is not mandatory, the possibility of including an additional bias term exists. In this way, improvements can be made in prediction performance by reducing generalization error. They can be incorporated within the branch network, and also an additional scalar bias $b_0 \in \mathbb{R}$ can also be added to the final DeepONet output, yielding in

$$G_{\boldsymbol{\theta}}(u)(\boldsymbol{\xi}_i) = \sum_{k=1}^{q} \beta_k(u) \cdot \tau_k(\boldsymbol{\xi}_i) + b_0, \quad u \in \mathcal{U}, \quad \boldsymbol{\xi}_i \in K_2, \quad b_0 \in \mathbb{R} \tag{26}$$

It is worth noting that, although DeepONet and other related neural operator frameworks, like Fourier Neural Operators (Li et al., 2021), POD-DeepONets (Lu et al., 2022), MIONetso (Jin et al., 2022), or DeepMMnets (Cai et al., 2021). While they have been applied to many multi-scale and multi-physics problems because they provide flexible tools for learning mappings between function spaces, they may not be the most optimal model in certain physical contexts. Models trained purely from data may still produce solutions that violate the governing equations. Particularly, when the available training data is limited, the network cannot adequately represent the underlying solution operator, or inaccuracies arise from the treatment of initial and boundary conditions, numerical discretization, or other modeling assumptions. For this reason, other models such as PI-DeepONet (Wang et al., 2021b) have been developed in recent years. This model introduces the physical knowledge into the training process by adding mean-squared penalties for violations of the governing PDE and its initial and boundary conditions to the total MSE loss function.

In the standard setting, the optimal vector of parameters $\boldsymbol{\theta}^*$ is learned by training DeepONet using a set of labeled observations. The parameters $\boldsymbol{\theta}$ of the DeepONet are obtained by minimizing the Mean Squared Error (MSE), which is a loss function that measures the discrepancy between the predicted optimal vector of trained network parameters and the corresponding numerical solutions of our solver. The standard MSE is defined as $\mathrm{MSE} = \frac{1}{N}\sum_{i=1}^{N}(y_i - \hat{y}_i)^2$, where y_i denotes the reference value and $\hat{y}_i$ the corresponding model prediction. As mentioned before, the model predicts the free-surface elevation $\zeta(x,t)$ and the depth-averaged velocity $u(x,t)$. Therefore, the optimization problem in our setting can be written as

$$\boldsymbol{\theta}^* = \arg\min_{\boldsymbol{\theta}} \mathcal{L}_{\mathrm{data}}(\boldsymbol{\theta}), \tag{27}$$

where

$$\mathcal{L}_{\mathrm{data}} = \frac{1}{N}\sum_{i=1}^{N}\left[\left(\zeta_i - \hat{\zeta}_i(\boldsymbol{\theta})\right)^2 + \left(u_i - \hat{u}_i(\boldsymbol{\theta})\right)^2\right] \tag{28}$$

Here, ζ_i and u_i denote the reference free-surface elevation and depth-averaged velocity obtained from the numerical one-dimension shallow-water PDE solver over the parameterized channel, while $\hat{\zeta}_i(\boldsymbol{\theta})$ and $\hat{u}_i(\boldsymbol{\theta})$ are the corresponding DeepONet predictions. The parameter N indicates the total number of training samples.

V. Case Study

A. Physical Domain and Numerical Configuration

The purpose of this work is to investigate whether PINN and DeepONet can serve as surrogate methods for parametrized and customizable coastal-flow ADCIRC-type domains. This serves as a preliminary step toward applications involving more complex phenomena, such as storm surges and tsunamis.

To this end, we have decided to create an idealized coastal problem using a rectangular open channel of length $L = 10,000\mathrm{m}$, and constant width $b = 1\mathrm{m}$, thus using a one-dimensional space-time domain $\Omega = \Omega_x \times (0,T)$, with T being the total period of the simulation and $\Omega_x = (0,L)$ the spatial domain defined between the open-sea ($x = 0$) and closed-wall ($x = L$) boundaries.

Furthermore, we discretize the mesh using uniformly equally spaced gridpoints with a distance of $\Delta x = 100$ meters, which divides the domain into $N = 100$ uniformly-spaced finite-volume cells. On the other hand, we have imposed a reference still-water depth of $H_0 = 10\mathrm{m}$. Initially, the free-surface elevation ζ and depth-averaged velocity u satisfy the initial conditions

$$\zeta(x,0) = 0, \qquad u(x,0) = 0, \tag{29}$$

so that the initial local water depth is

$$h(x,0) = H_0 - z_b(x) \tag{30}$$

With the goal of simulating an idealized coastal profile, we have chosen to impose a non-flat bathymetry in our setup that begins with zero bed elevation from $x = 0$ to x_{flat} and increases linearly to reach the prescribed value $\mathrm{bed_rise}$ at $x = L$. Although more complex bathymetries could be considered in future work, this piecewise-linear configuration provides a controlled setting to test the performance of both surrogate models in channels with spatially varying bed elevation. We have also used the following function $z_b(x)$, which describes the bed elevation at each discretized point x of the bathymetry:

$$z_b(x) = \begin{cases} 0, & x < x_{\mathrm{flat}}, \\ \mathrm{bed_rise}\,\frac{x - x_{\mathrm{flat}}}{L - x_{\mathrm{flat}}}, & x > x_{\mathrm{flat}}, \end{cases} \tag{31}$$

B. Training Dataset

In this work, we have decided to use a total of 20 independently generated scenarios, varying the tidal amplitude, wind speed, and bathymetric rise within specific prescribed uniform distributions. The ability of this relatively small ensemble for generalization is subsequently assessed by comparing its results with a previously unseen scenario, whose forcing and bathymetric parameters are randomly sampled from within the ranges used to generate the training dataset. Dirichlet boundary conditions are imposed at both ends of the channel. At the open-ocean boundary $x = 0$, the free-surface elevation is

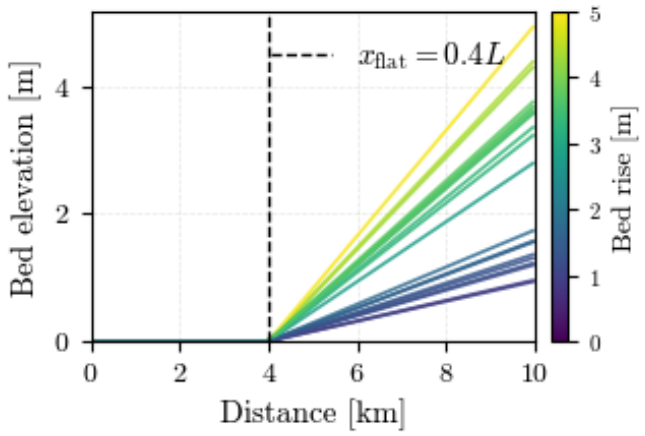


Fig. 4: Bathymetric profiles of the 20 independently generated training scenarios. In each one, the bed is flat on the horizontal axis until x_{flat}, where it increases linearly to a randomly sampled final bed elevation from the interval $[1, 5]$ m at $x = L$. This variation allows both surrogate models to be trained across different physical configurations.

imposed through a multi-constituent tidal signal, $\zeta(0, t) = \zeta_{\text{tide}}(t)$. The tidal amplitude physical parameter A_{tide} is randomly chosen from $A_{\text{tide}} \in [0.5, 2.5]$ m for each scenario and is treated as an input parameter, whereas the tidal constituents and their relative amplitudes, periods, and phases remain fixed across the samples. At the right end of the channel, $x = L$, a closed-wall boundary condition to the depth-averaged velocity is imposed by enforcing $u(L, t) = 0$. The wind speed is assumed to be spatially uniform throughout the channel and varies within the interval $U_{\text{wind}} \in [0, 40]\,\text{m s}^{-1}$. It is important to note that both tidal and wind forcings are introduced gradually over a startup period of $T_{\text{ramp}} = 3600\,\text{s}$ to avoid their abrupt application at $t = 0$. For each sample, the bathymetry parameter `bed_rise` is chosen from the interval $[0, 5]$ m. As we said before, the bed remains flat until $x_{\text{flat}} = 0.4L$, where it rises linearly until `bed_rise` at $x = L$. Bottom-friction effects are represented using Manning's formulation with a roughness coefficient of $n_M = 0.02\,\text{s m}^{-1/3}$. Finally, due to numerical stability reasons, the numerical solver advances the solution with a time step of $\Delta t = 2$s over a total simulation period of $T_{\text{total}} = 48$h.

C. DeepONet Training Configuration

In the context of this work, we detail below the mathematical application of the DeepONet model. First, we discretize the initial channel into a set of spatial points. Subsequently, we select m sensors from this initial discretization and evaluate them using the bathymetry function (31), which corresponds to the function v in our explanation of DeepONet, to obtain the bed elevation at those sensor locations. The resulting sensor vector is augmented with the two scalar phyisical parameters considered in this case: the tidal amplitude a_{tide}, which parameterizes the time-dependent free-surface elevation prescribed at the open-ocean boundary $x = 0$, and the wind speed U_{wind}, which is applied uniformly throughout the channel. Consequently, the physical-parameter vector is $\boldsymbol{\mu} = (A_{\text{tide}}, U_{\text{wind}})^{\mathsf{T}}$, making $r = 2$, and defining a finite-dimensional representation of the input function: $\gamma(v) := [\text{w}_{\text{speed}}, A_{\text{tide}}, v(\tilde{x}_1), v(\tilde{x}_2), \ldots, v(\tilde{x}_m)]$. Then, the branch network defines a mapping $\boldsymbol{\beta} : \mathbb{R}^{r+m} \to \mathbb{R}^{2p}$ using a neuronal network, and produces two sets of p-dimensional latent coefficients, $\boldsymbol{\beta}^{\zeta}$ and $\boldsymbol{\beta}^{u}$, associated with the free-surface elevation ζ and depth-averaged velocity u of the PDE system defined. In parallel, the trunk network receives the spatio-temporal query coordinates $\boldsymbol{\xi}_i = (x_i, t_i) \in K_2 \subset \mathbb{R}^2$ and maps it through $\boldsymbol{\tau} : K_2 \to \mathbb{R}^p$ to the latent basis vector $\boldsymbol{\tau}(\boldsymbol{\xi}_i)$. After that, two independent inner products are computed between the trunk representation and the corresponding branch coefficients, with the addition of the trainable biases b_ζ and b_u, yielding the predictions $\hat{\zeta}_{\boldsymbol{\theta}}(\boldsymbol{\xi}_i; \boldsymbol{\mu}, z_b) = \boldsymbol{\beta}^{\zeta}(\boldsymbol{\mu}, z_b) \cdot \boldsymbol{\tau}(\boldsymbol{\xi}_i) + b_\zeta$, and $\hat{u}_{\boldsymbol{\theta}}(\boldsymbol{\xi}_i; \boldsymbol{\mu}, z_b) = \boldsymbol{\beta}^{u}(\boldsymbol{\mu}, z_b) \cdot \boldsymbol{\tau}(\boldsymbol{\xi}_i) + b_u$. These predictions form the operator output $\mathcal{G}_{\boldsymbol{\theta}}(\boldsymbol{\mu}, z_b)(\boldsymbol{\xi}_i) = \left(\hat{\zeta}_{\boldsymbol{\theta}}(\boldsymbol{\xi}_i; \boldsymbol{\mu}, z_b), \hat{u}_{\boldsymbol{\theta}}(\boldsymbol{\xi}_i; \boldsymbol{\mu}, z_b)\right)$. The model parameters $\boldsymbol{\theta}$ are learned by minimizing the supervised data loss $\mathcal{L}_{\text{data}}(\boldsymbol{\theta})$ against to numerical 1D shallow-water equation solutions. The model is trained together with the numerical solutions that produce the one-dimensional shallow-water PDE solver that we have programmed. Therefore, we apply (28) to calculate the supervised data loss. Subsequently, the resulting gradients and biases are used to update the parameters of both the branch and trunk networks through gradient-based optimization until reaching the prescribed number of epochs, producing the trained parameter vector

$$\boldsymbol{\theta}^* = \arg\min_{\boldsymbol{\theta}} \left\{ \frac{1}{N_{\text{xt}}} \sum_{i=1}^{N_{\text{xt}}} \left[\left(\hat{\zeta}_{\boldsymbol{\theta}}(\boldsymbol{\xi}_i; \boldsymbol{\mu}_i, z_{b,i}) - \zeta_i \right)^2 + \left(\hat{u}_{\boldsymbol{\theta}}(\boldsymbol{\xi}_i; \boldsymbol{\mu}_i, z_{b,i}) - u_i \right)^2 \right] \right\} \tag{32}$$

D. PINN Training Configuration

The PINN model applied in our work shares some similarities with DeepONet. First, only a single feedforward neural network is employed, into which the following input vector is fed

$$\left\{ \boldsymbol{\chi}_i^{(k)} \right\}_{i=1}^{N_k}, \qquad k \in \{\text{data, phys, IC, BC}\}, \tag{33}$$

The physical parameter vector $\boldsymbol{\mu}$ is composed of the wind speed, the tide amplitude at $x = 0$, and the `bed_rise` parameter we introduced before. These are primary physical parameters in this work, so $r = 3$. Thus, the PINN defines a mapping $NN_{\boldsymbol{\theta}} : \mathbb{R}^5 \to \mathbb{R}^2$, where the initial vector is:

$$\boldsymbol{\chi}_i^{(k)} = \begin{pmatrix} x_i^{(k)} \\ t_i^{(k)} \\ \text{w}_{\text{speed}_i}^{(k)} \\ A_{\text{tide}_i}^{(k)} \\ \text{bed_rise}_i^{(k)} \end{pmatrix} \in \mathbb{R}^5 \tag{34}$$

After going through the hidden layers of the neural network, the network returns the approximation $\hat{\mathbf{y}}_{\boldsymbol{\theta}}(x, t; \boldsymbol{\mu}) = \left(\hat{\zeta}_{\boldsymbol{\theta}}(x, t; \boldsymbol{\mu}), \hat{u}_{\boldsymbol{\theta}}(x, t; \boldsymbol{\mu})\right)^{\mathsf{T}} \in \mathbb{R}^2$. Then, the derivatives of these two output variables are computed with respect to x and

t through automatic differentiation, an important step to continue with the computation of the continuity and momentum residuals $\mathcal{R}_c$ and $\mathcal{R}_m$, respectively. The estimated total water depth is given by $\hat{h}_{\boldsymbol{\theta}}(x,t;\boldsymbol{\mu}) = H_0 + \hat{\zeta}_{\boldsymbol{\theta}}(x,t;\boldsymbol{\mu}) - z_b(x;\boldsymbol{\mu})$, whereas the momentum source $\mathcal{S}_m$ incorporates the forcing, bed-slope, and friction contributions included in the setting. After this step, training of the neural network follows, and it is guided by four error terms: the initial-condition loss $\mathcal{L}_{\text{IC}}$, the boundary-condition loss $\mathcal{L}_{\text{BC}}$, the supervised data loss $\mathcal{L}_{\text{data}}$, and the equation-residual loss $\mathcal{L}_{\text{phys}}$. All are computed using the MSE, yielding the following total data loss:

$$\mathcal{L}_{\text{IC}}(\boldsymbol{\theta}) = \text{MSE}\left(\hat{\mathbf{y}}_{\boldsymbol{\theta}}(x_{\text{IC}},0), \mathbf{y}(x_{\text{IC}},0)\right) \tag{35}$$

$$\begin{aligned}\mathcal{L}_{\text{BC}}(\boldsymbol{\theta}) = \text{MSE}&\left(\hat{\zeta}_{\boldsymbol{\theta}}(0,t_{\text{BC}}), \zeta_{\text{BC}}(t_{\text{BC}})\right)\\ &+ \text{MSE}\left(\hat{u}_{\boldsymbol{\theta}}(L,t_{\text{BC}}), u_{\text{BC}}(t_{\text{BC}})\right)\end{aligned} \tag{36}$$

$$\mathcal{L}_{\text{data}}(\boldsymbol{\theta}) = \text{MSE}\left(\hat{\mathbf{y}}_{\boldsymbol{\theta}}(x_{\text{data}},t_{\text{data}}), \mathbf{y}(x_{\text{data}},t_{\text{data}})\right) \tag{37}$$

$$\begin{aligned}\mathcal{L}_{\text{phys}}(\boldsymbol{\theta}) = \text{MSE}&\left(\mathcal{R}_{c,\boldsymbol{\theta}}(x_{\text{phys}},t_{\text{phys}}), 0\right)\\ &+ \text{MSE}\left(\mathcal{R}_{m,\boldsymbol{\theta}}(x_{\text{phys}},t_{\text{phys}}), 0\right)\end{aligned} \tag{38}$$

Recall that mean square error (MSE) between a predicted value $\hat{\mathbf{y}}$ and its target value $\mathbf{y}$ over N sampled points is defined as

$$\text{MSE}\left(\hat{\mathbf{y}}, \mathbf{y}\right) = \frac{1}{N}\sum_{i=1}^{N}\left\|\hat{\mathbf{f}}(\boldsymbol{\chi}_i) - \mathbf{f}(\boldsymbol{\chi}_i)\right\|_2^2 \tag{39}$$

Now, the total data loss is computed using the weighted sum (22) in order to balance the relative contribution of each loss during training. The ground-truth values $\mathbf{y}(x,t;\boldsymbol{\mu}) = (\zeta(x,t;\boldsymbol{\mu}), u(x,t;\boldsymbol{\mu}))^{\mathsf{T}}$ required for the data loss are calculated using the PDE numerical solver implemented in this work for the 1D shallow-water system. Finally, the gradient-based optimization adjusts the network parameters $\boldsymbol{\theta}$ and biases until reaching the prescribed number of epochs, producing the trained parameter vector

$$\begin{aligned}\boldsymbol{\theta}^*_{\text{PINN}} = \arg\min_{\boldsymbol{\theta}}\mathcal{L}_{\text{total}}(\boldsymbol{\theta}) = \arg\min_{\boldsymbol{\theta}}[\lambda_{\text{IC}}\mathcal{L}_{\text{IC}}(\boldsymbol{\theta}) +\\ +\lambda_{\text{BC}}\mathcal{L}_{\text{BC}}(\boldsymbol{\theta}) + \lambda_{\text{data}}\mathcal{L}_{\text{data}}(\boldsymbol{\theta}) + \lambda_{\text{phys}}\mathcal{L}_{\text{phys}}(\boldsymbol{\theta})]\end{aligned} \tag{40}$$

Lastly, it is important to mention that in order to proceed with the computation of the physics-loss component of the PINN, we need to compute the residuals of both the continuity and momentum parts. From the vector representation of the one-dimensional shallow-water equations in 1, we can define the the equation embedded in the first row as the *continuity equation*, whereas the one in the second row is the *momentum equation*. Following the vector formulation from 1, these residuals can be collected into the vector

$$\begin{aligned}\mathcal{R}_{\boldsymbol{\theta}}(x,t;\boldsymbol{\mu}) = \frac{\partial \hat{U}_{\boldsymbol{\theta}}(x,t;\boldsymbol{\mu})}{\partial t} + \frac{\partial F\left(x, \hat{U}_{\boldsymbol{\theta}}(x,t;\boldsymbol{\mu})\right)}{\partial x}\\ - H\left(x, \hat{U}_{\boldsymbol{\theta}}(x,t;\boldsymbol{\mu})\right)\end{aligned} \tag{41}$$

Equivalently, the residual vector can be written as

$$\mathcal{R}_{\boldsymbol{\theta}}(x,t;\boldsymbol{\mu}) = \begin{pmatrix}\mathcal{R}_{c,\boldsymbol{\theta}}(x,t;\boldsymbol{\mu})\\ \mathcal{R}_{m,\boldsymbol{\theta}}(x,t;\boldsymbol{\mu})\end{pmatrix} \tag{42}$$

where $\mathcal{R}_{c,\boldsymbol{\theta}}$ and $\mathcal{R}_{m,\boldsymbol{\theta}}$ denote the continuity and momentum residuals equation. optimization-based parameters θ. Entry-wise, these residuals are computed as

$$\mathcal{R}_{c,\boldsymbol{\theta}} = \frac{\partial \hat{A}_{\boldsymbol{\theta}}}{\partial t} + \frac{\partial \hat{Q}_{\boldsymbol{\theta}}}{\partial x}, \tag{43}$$

$$\begin{aligned}\mathcal{R}_{m,\boldsymbol{\theta}} = \frac{\partial \hat{Q}_{\boldsymbol{\theta}}}{\partial t} + \frac{\partial}{\partial x}\left(\frac{\hat{Q}^2_{\boldsymbol{\theta}}}{\hat{A}_{\boldsymbol{\theta}}} + g\hat{I}_{1,\boldsymbol{\theta}}\right)\\ - g\left[I_2 + \hat{A}_{\boldsymbol{\theta}}\left(S_0 - \hat{S}_{f,\boldsymbol{\theta}}\right)\right]\end{aligned} \tag{44}$$

Since our problem has a constant rectangular channel width b, 7 simplifies the wet cross-sectional area to $\hat{A}_{\boldsymbol{\theta}} = b\hat{h}_{\boldsymbol{\theta}}$, which also yields the volumetric water discharge to be $\hat{Q}_{\boldsymbol{\theta}} = b\hat{h}_{\boldsymbol{\theta}}\hat{u}_{\boldsymbol{\theta}}$. Our channel have constant width b, so $\sigma(x,\eta) = b$ along the whole spatial-temporal domain. Under the same constant-width assumption, the hydrostatic pressure integral defined in (11) simplifies to

$$I_1(x) = \frac{1}{2}bh^2 \tag{45}$$

This width fact also makes the pressure force source integral from (12) to vanish:

$$I_2(x) = 0 \tag{46}$$

As for the calculation of the shallow water equations we need the bed slope S_0 from (10), then we simply derive (31) with respect to x, yielding the following piecewise formula:

$$S_0(x) = \begin{cases}0, & x < x_{\text{flat}},\\ -\dfrac{\text{bed_rise}}{L - x_{\text{flat}}}, & x > x_{\text{flat}}.\end{cases} \tag{47}$$

Furthermore, the friction slope S_f associated with momentum loss, defined in (9), must be adjusted for our case. As previously mentioned, it is evaluated using Manning's formulation:

$$\hat{S}_{f,\boldsymbol{\theta}}(x,t;\boldsymbol{\mu}) = \frac{n_M^2\hat{u}_{\boldsymbol{\theta}}(x,t;\boldsymbol{\mu})\left|\hat{u}_{\boldsymbol{\theta}}(x,t;\boldsymbol{\mu})\right|}{\hat{R}^{4/3}_{h,\boldsymbol{\theta}}(x,t;\boldsymbol{\mu})}, \tag{48}$$

where n_M is Manning's roughness coefficient and $\hat{R}_{h,\boldsymbol{\theta}}$ is the predicted hydraulic radius. This radius is defined as the ratio between the wetted cross-sectional area $\hat{A}_{\boldsymbol{\theta}}$ and the wetted perimeter $\hat{P}_{\boldsymbol{\theta}}$:

$$\hat{R}_{h,\boldsymbol{\theta}} = \frac{\hat{A}_{\boldsymbol{\theta}}}{\hat{P}_{\boldsymbol{\theta}}}. \tag{49}$$

For the rectangular channel without restrictions at the top considered in our sample case, $\hat{A}_{\boldsymbol{\theta}} = b\hat{h}_{\boldsymbol{\theta}}$ and $\hat{P}_{\boldsymbol{\theta}} = b + 2\hat{h}_{\boldsymbol{\theta}}$. Then,

$$\hat{R}_{h,\boldsymbol{\theta}} = \frac{b\hat{h}_{\boldsymbol{\theta}}}{b + 2\hat{h}_{\boldsymbol{\theta}}} \tag{50}$$

With this assumptions, the continuity and momentum residual equation can then be reformulated for our specific channel:

$$\mathcal{R}_{c,\boldsymbol{\theta}} = \frac{\partial \hat{\zeta}_{\boldsymbol{\theta}}}{\partial t} + \frac{\partial}{\partial x}\left(\hat{h}_{\boldsymbol{\theta}}\hat{u}_{\boldsymbol{\theta}}\right), \tag{51}$$

$$\mathcal{R}_{m,\boldsymbol{\theta}} = \frac{\partial}{\partial t}\left(\hat{h}_{\boldsymbol{\theta}}\hat{u}_{\boldsymbol{\theta}}\right) + \frac{\partial}{\partial x}\left(\hat{h}_{\boldsymbol{\theta}}\hat{u}_{\boldsymbol{\theta}}^2 + \frac{g}{2}\hat{h}_{\boldsymbol{\theta}}^2\right) - g\hat{h}_{\boldsymbol{\theta}}\left(S_0 - \frac{n_M^2 \hat{u}_{\boldsymbol{\theta}}\left|\hat{u}_{\boldsymbol{\theta}}\right|}{\left(\frac{b\hat{h}_{\boldsymbol{\theta}}}{b+2\hat{h}_{\boldsymbol{\theta}}}\right)^{4/3}}\right) - \frac{\tau_{\text{wind}}}{\rho_{\text{water}}} \tag{52}$$

$$\frac{\tau_{\text{wind}}(t;\boldsymbol{\mu})}{\rho_{\text{water}}} = \frac{\rho_{\text{air}} C_D U_{\text{wind}}^2}{\rho_{\text{water}}} r(t) \tag{53}$$

where ρ_{air} is the air density, ρ_{water} is the water density, C_D is the wind-drag coefficient and U_{wind} is the prescribed wind speed. As we said before, to avoid an abrupt introduction of wind forcing at $t = 0$, we introduced a ramp-time function $r(t)$, which increases linearly from zero to one over a prescribed ramp time T_{ramp}:

$$r(t) = \begin{cases} \dfrac{t}{T_{\text{ramp}}}, & 0 \le t < T_{\text{ramp}}, \\ 1, & t \ge T_{\text{ramp}}. \end{cases} \tag{54}$$

VI. Results

In the following section we explain the results after training the two surrogate models on the 20 independently generated scenarios described previously, using 8000 epochs and the hyperbolic tangent as the activation function. Training was performed using the Adam gradient-based optimizer (Kingma and Ba, 2015) with a learning rate of 0.001, based on preliminary experiments to balance convergence speed and stability. Also, both models were trained on a fine grid of $N_c = 1000$ collocation points and using three hidden layers with 64 nodes per layer.

A. Training Performance

To evaluate scalability and efficiency of both surrogate models, TABLE I reports the training time of both PINN and DeepONet, which denotes the time the surrogate models took to be trained with the 20 training scenarios, along with the inference time, which is the time required to predict a scenario with 20,000 spatio-temporal points, but with the forcing and bathymetric input parameters equal to zero. The inference time can also can be understood as the time to make one prediction (i.e, a forward run of the whole network). All measures where done using the same hardware.

The results in TABLE I show that DeepONet's inference time is 2.1 times greater than that of PINN, indicating that the latter makes a prediction faster. A reason that may explain this difference is that DeepONet has a more complex architecture. While PINN is described with a single neural network that jointly takes spatial-temporal coordinates and phyiscal parameters and maps them directly onto the solution manifold, the DeepONet requires separate evaluations of the branch and trunk networks followed by a combination of their latent representations, which introduces additional computational operations while making an inference. On the other hand, the training time of PINN is 1.5 times higher than that of DeepONet. In this case, this difference can be attributed, in part, to the fact that in the PINN there is an additional computational cost associated with the physics-informed loss parts. In particular, the evaluation of the PDE residuals requires computing the spatial and temporal partial derivatives through automatic differentiation at the collocation points, which increases the cost of each optimization step. Recall that DeepONET does not involve in this and seek to learn a mapping between the input and output function spaces.

TABLE I: Comparison of the computational time between the PINN and DeepONet for a batch of 20,000 space-time points.

Model	Training time [s]	Inference time [ms]
PINN	1063.0	16.8
DeepONet	707.0	35.2

B. Predictions on Training Scenarios

Since the magnitude of SME in (39) is in squared units of the target variable y_i, metrics like the Root Mean Squared Error (RMSE) are often preferred for interpret and evaluate models because it is expressed in the same physical units as the predicted variable. For n data points $\{(y_i, \hat{y}_i)\}_{i=1}^n$, where y_i is the true value and $\hat{y}_i$ is the predicted value, the Root Mean Squared Error (RMSE) is given by

$$\text{RMSE} = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2} \tag{55}$$

The dot plots presented in Fig 5 illustrates the RMSE for the free-surface elevation and depth-averaged velocity in each training scenario. This visualization offers a clear comparative analysis of the predictive performance of each model in each training scenario. However, for both state variables, that performance from both surrogate models is strongly scenario-dependent and neither uniformly outperforms the other, with the RMSE varying by up to 0.10 points across the different forcing and bathymetry configurations in each model.

C. Generalization to an Unseen Scenario

In order to show that the training models can be generalized to scenarios not seen during the training process rather than memorizing the training set, we generated an independently unseen random scenario using the same physical domain and numerical configuration as the previous scenarios used in the training process. In this way, we can evaluate whether both models face the overfitting problem, in which the low errors obtained on the training set are preserved on the random scenario. Overfitting is a very recurring problem in deep-learning based physics applications and has been widely discussed in

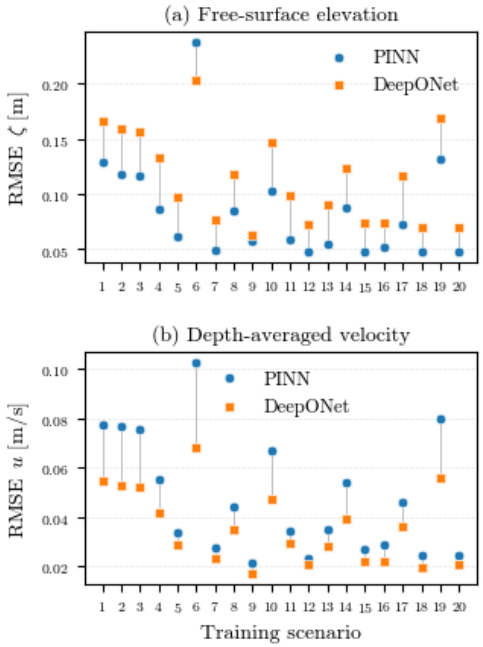


Fig. 5: Space-time final RMSE of the PINN and DeepONet predictions across the 20 training scenarios for (a) free-surface elevation and (b) depth-averaged velocity after the training process.

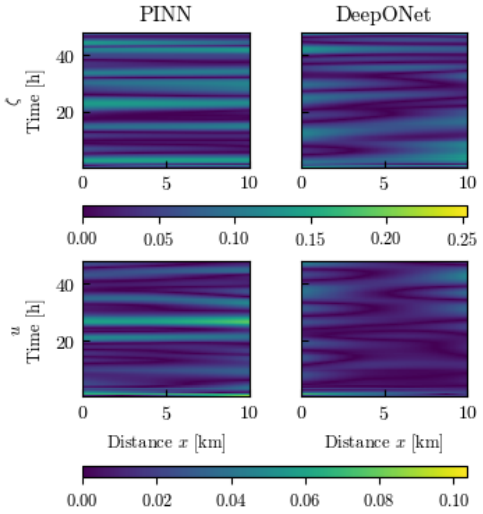


Fig. 6: Space-time absolute-error heatmap of both the predictions from the PINN and DeepONet models for the randomly generated unseen scenario whose forcing and bathymetric parameters lie within the ranges used for generating the training scenarios.

a large number of studies, such as the one presented in Bajaj et al. (2023). In Fig 6 we show an absolute-error heatmap of each model, where it is visible that the absolute errors remain below $0.25\,\mathrm{m}$ for the free-surface elevation and $0.10\,\mathrm{m\,s^{-1}}$ for the depth-averaged velocity, demonstrating that both models can learn the behavior of the underlying PDEs.

D. PINN and DeepONet Comparison

In Fig. 7 we show the evolution of the data loss of both models over the training epochs. We can observe that both models show a steady decline and eventually reaches a plateau, which suggests that the learning processes of both converge. However, the DeepONet show a greater fluctuation (or spikes) than the PINN, suggesting a less stable optimization process during training. During the early stages of training, the PINN achieve a lower training data loss than the DeepONet, indicating a greater initial reduction of the supervisory predictive error. Despite these frequent fluctuations, the overall trend of the loss from both models is downward as the number of epoch increases. It is important to note that we just included the training data loss to provide a common reference in the comparison between both models.

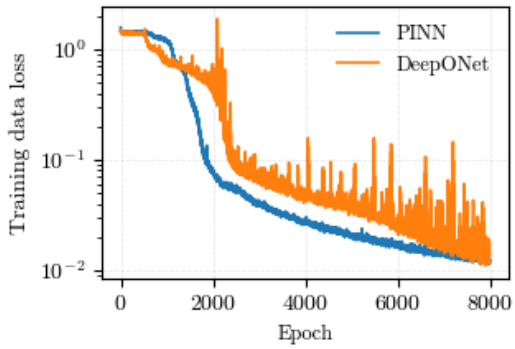


Fig. 7: Evolution of the supervised training-data loss of both the PINN and DeepONet surrogate models over the traning epochs.

To further assess the unseen scenario, in Fig. 9 we used the empirical Cumulative Distribution Function (CDF) as a metric to evaluate the accuracy of the models behavior against the unseen scenario. The empirical CDF is defined as

$$\hat{F}_E(e) = \frac{1}{N}\sum_{i=1}^{N} \mathbf{1}(E_i \leq e), \tag{56}$$

where N denotes the total number of space-time points, $\mathbf{1}$ is the indicator function, E_i is the absolute error at the $i-th$ space-time point, and e is a prescribed error threshold. This means that $\hat{F}_E(e)$ represents the fraction of spatio-temporal points with absolute error, $E_{\zeta,i} = \left|\zeta_i - \hat{\zeta}_i\right|$ and $E_{u,i} = |u_i - \hat{u}_i|$, less than a given threshold. Recall that ζ_i and u_i are the references values and $\hat{\zeta}_i$ and $\hat{u}_i$ are the respective models predicted values. It can be observed in the figure that DeepONet slightly outperforms PINN in both free-surface elevation and depth-averaged flow velocity predictions for the considered random unseen scenario since its curve lies above and to the left of the PINN curve.

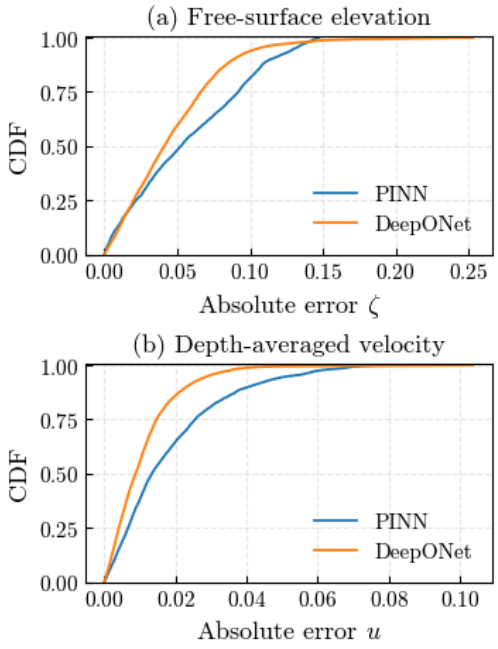


Fig. 9: Empirical Cumulative Distribution Functions (CDFs) of the pointwise absolute errors produced by the PINN and DeepONet surrogate models predicting the (a) free-surface elevation, and (b) depth-averaged velocity of the randomly generated unseen scenario.

Beyond predictive accuracy with respect to the reference data, it is also important to assess the physical consistency of both surrogate models, which is commonly evaluated through the residuals of the governing PDEs. Recall that the residuals measures how much the predicted solutions satisfy the underlying PDE equations. In TABLE II shown the results we got after having evaluated $4,000$ randomly sampled collocation points under a flat bathymetry domain using the norm of both the continuity and momentum residuals formulas defined on (51) and (52). The PINN surrogate model achieved a lower residual PDE norm than the DeepONet in both residuals, achieving 78.6% and 82% lower residual continuity and momentum, respectively. Unlike DeepONet, the PINN includes the continuity and momentum residuals (i.e., PDE residuals) in the loss function, which are therefore directly minimized during the training process. However, the conventional DeepONet used here is trained solely on paired input-output data from each scenario and does not enforce the governing shallow-water PDEs. Although DeepONet may accurately approximate the reference solutions, this does not necessarily imply that the governing PDEs are only minimally violated.

TABLE II: Physics-residual comparison evaluated at $4{,}000$ randomly sampled collocation points under a flat bathymetric profile. Lower values indicate a better satisfaction to the governing 1D shallow-water equations.

Residual	PINN	DeepONet
Continuity $\lvert\mathcal{R}_c\rvert \downarrow$	$\mathbf{2.9526 \times 10^{-1}}$	1.3805×10^{0}
Momentum $\lvert\mathcal{R}_m\rvert \downarrow$	$\mathbf{3.1573 \times 10^{-1}}$	1.7520×10^{0}

VII. Future Work

Future work could involve expanding to include other forcing conditions, such as storms that move across the domain,

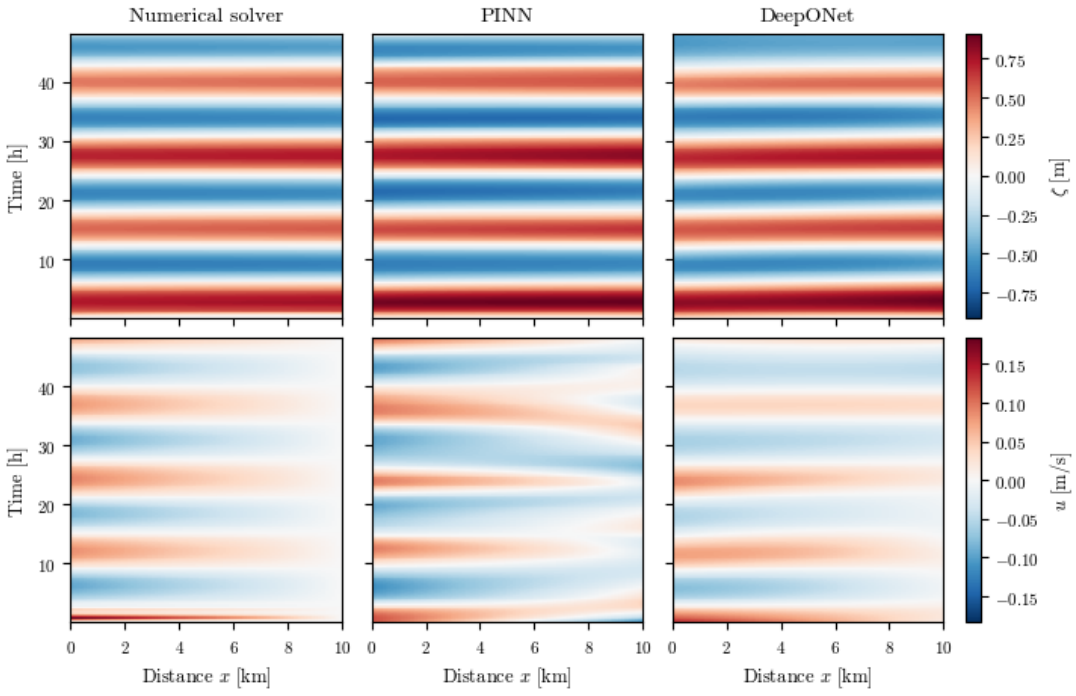


Fig. 8: Space-time heatmaps of the free-surface elevation (top row) and depth-averaged velocity (bottom row) for the unseen scenario. The columns show the numerical reference solution, PINN prediction, and DeepONet prediction, respectively.

along with more complex bathymetric profiles, which could provide a more accurate representation of different phenomena. Furthermore, these customized channels could continue to be used along with Physics-Informed Deep Operational Networks (PI-DeepONet), which extend the standard DeepONet framework to satisfy underlying physical laws by minimizing the physical residual. This is achieved by incorporating the residuals of the governing shallow-water PDEs into the training loss, following the same principle used in PINNs.

VIII. Code and model availability

The source code used to train and evaluate the PINN and DeepONet models, together with the numerical solver used to generate the reference scenarios and their respective solutions, are available to the public in the following link: `github.com/rhogas/Shallow-Water-Surrogate`.

References

Chandrajit Bajaj, Luke McLennan, Timothy Andeen, and Avik Roy. Recipes for when physics fails: recovering robust learning of physics informed neural networks. Machine Learning: Science and Technology, 4(1):015013, 2023. doi: 10.1088/2632-2153/acb416.

Natacha B. Bernier and Keith R. Thompson. Deterministic and ensemble storm surge prediction for atlantic canada with lead times of hours to ten days. Ocean Modelling, 86:114–127, 2015. doi: 10.1016/j.ocemod.2014.12.002. URL https://doi.org/10.1016/j.ocemod.2014.12.002.

Shengze Cai, Zhicheng Wang, Lu Lu, Tamer A. Zaki, and George Em Karniadakis. DeepM&Mnet: Inferring the electroconvection multiphysics fields based on operator approximation by neural networks. Journal of Computational Physics, 436:110296, 2021. doi: 10.1016/j.jcp.2021.110296. URL https://doi.org/10.1016/j.jcp.2021.110296.

Tianping Chen and Hong Chen. Universal approximation to nonlinear operators by neural networks with arbitrary activation functions and its application to dynamical systems. IEEE Transactions on Neural Networks, 6(4):911–917, 1995. doi: 10.1109/72.392253. URL https://doi.org/10.1109/72.392253.

Stéphane Clain and Jorge Figueiredo. The MOOD method for the non-conservative shallow-water system. Computers & Fluids, 145:99–128, 2017. doi: 10.1016/j.compfluid.2016.11.013. URL https://doi.org/10.1016/j.compfluid.2016.11.013.

George Cybenko. Approximation by superpositions of a sigmoidal function. Mathematics of Control, Signals and Systems, 2:303–314, 1989. doi: 10.1007/BF02551274.

J. C. Dietrich, M. Zijlema, J. J. Westerink, L. H. Holthuijsen, C. Dawson, R. A. Luettich, R. E. Jensen, J. M. Smith, G. S. Stelling, and G. W. Stone. Modeling hurricane waves and storm surge using integrally-coupled, scalable computations. Coastal Engineering, 58(1):45–65, 2011. doi: 10.1016/j.coastaleng.2010.08.001. URL https://doi.org/10.1016/j.coastaleng.2010.08.001.

M. W. M. G. Dissanayake and N. Phan-Thien. Neural-network-based approximations for solving partial differential equations. Communications in Numerical Methods in Engineering, 10(3):195–201, 1994. doi: 10.1002/cnm.1640100303.

Kerry Emanuel. Increasing destructiveness of tropical cyclones over the past 30 years. Nature, 436:686–688, 2005. doi: 10.1038/nature03906. URL https://doi.org/10.1038/nature03906.

Stefanos Giaremis, Noujoud Nader, Clint Dawson, Carola Kaiser, Efstratios Nikidis, and Hartmut Kaiser. Storm surge modeling in the AI era: Using LSTM-based machine learning for enhancing forecasting accuracy. Coastal Engineering, 191:104532, 2024. doi: 10.1016/j.coastaleng.2024.104532. URL https://doi.org/10.1016/j.coastaleng.2024.104532.

Don Hinrichsen. Coastal Waters of the World: Trends, Threats, and Strategies. Island Press, Washington, DC, 1998. URL https://books.google.com/books?id=70NsJkGMPiwC.

Ben R. Hodges. Conservative finite-volume forms of the Saint-Venant equations for hydrology and urban drainage. Hydrology and Earth System Sciences, 23:1281–1304, 2019. doi: 10.5194/hess-23-1281-2019. URL https://doi.org/10.5194/hess-23-1281-2019.

M. E. Hope, J. J. Westerink, A. B. Kennedy, P. C. Kerr, J. C. Dietrich, C. Dawson, C. J. Bender, J. M. Smith, R. E. Jensen, M. Zijlema, L. H. Holthuijsen, R. A. Luettich, M. D. Powell, V. J. Cardone, A. T. Cox, H. Pourtaheri, H. J. Roberts, J. H. Atkinson, S. Tanaka, H. J. Westerink, and L. G. Westerink. Hindcast and validation of hurricane ike (2008) waves, forerunner, and storm surge. Journal of Geophysical Research: Oceans, 118(9):4424–4460, 2013. doi: 10.1002/jgrc.20314. URL https://doi.org/10.1002/jgrc.20314.

Junqin Hou, Muhammad K. Akbar, Manar D. Samad, and Lizhi Ouyang. Spatiotemporal deep learning to forecast storm surge water levels and storm trajectory: Case study hurricane harvey. Journal of Marine Science and Engineering, 13(9):1780, 2025. doi: 10.3390/jmse13091780. URL https://doi.org/10.3390/jmse13091780.

Ameya D. Jagtap, Ehsan Kharazmi, and George Em Karniadakis. Conservative physics-informed neural networks on discrete domains for conservation laws: Applications to forward and inverse problems. Computer Methods in Applied Mechanics and Engineering, 365:113028, 2020. doi: 10.1016/j.cma.2020.113028.

Pengzhan Jin, Shuai Meng, and Lu Lu. MIONet: Learning multiple-input operators via tensor product. SIAM Journal on Scientific Computing, 44(6):A3490–A3514, 2022. doi: 10.1137/22M1477751. URL https://doi.org/10.1137/22M1477751.

Martin A. Kern, Perry A. Bartelt, and Betty Sovilla. Velocity profile inversion in dense avalanche flow. Annals of Glaciology, 51(54):27–31, 2010. doi: 10.3189/172756410791386643. URL https://doi.org/10.3189/172756410791386643.

Ehsan Kharazmi, Zhongqiang Zhang, and George Em Kar-

niadakis. Variational physics-informed neural networks for solving partial differential equations. arXiv preprint arXiv:1912.00873, 2019. doi: 10.48550/arXiv.1912.00873.

Ehsan Kharazmi, Zhongqiang Zhang, and George Em Karniadakis. hp-vpinns: Variational physics-informed neural networks with domain decomposition. Computer Methods in Applied Mechanics and Engineering, 374:113547, 2021. doi: 10.1016/j.cma.2020.113547.

Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In International Conference on Learning Representations (ICLR), 2015. URL https://arxiv.org/abs/1412.6980.

Siva Srinivas Kolukula, P. L. N. Murty, T. Srinivasa Kumar, E. Pattabhi Ramarao, and M. V. Ramana Murthy. Tsunami modelling over global oceans. Royal Society Open Science, 12(1):241128, 2025. doi: 10.1098/rsos.241128. URL https://doi.org/10.1098/rsos.241128.

Alexander Kurganov, Mingye Na, and Vladimir V. Ostapenko. On conservative and nonconservative two-layer shallow water equations. SSRN Electronic Journal, 2025. doi: 10.2139/ssrn.5372483. URL https://doi.org/10.2139/ssrn.5372483. Preprint.

Thorne Lay, Hiroo Kanamori, Charles J. Ammon, Meredith Nettles, Steven N. Ward, Richard C. Aster, Susan L. Beck, Susan L. Bilek, Michael R. Brudzinski, Rhett Butler, Heather R. DeShon, Göran Ekström, Kenji Satake, and Stuart Sipkin. The great Sumatra–Andaman earthquake of 26 december 2004. Science, 308(5725):1127–1133, 2005. doi: 10.1126/science.1112250. URL https://doi.org/10.1126/science.1112250.

Zongyi Li, Nikola Kovachki, Kamyar Azizzadenesheli, Burigede Liu, Kaushik Bhattacharya, Andrew Stuart, and Anima Anandkumar. Fourier neural operator for parametric partial differential equations. In International Conference on Learning Representations. OpenReview.net, 2021. URL https://openreview.net/forum?id=c8P9NQVtmnO.

Ning Lin, James A. Smith, Gabriele Villarini, Timothy P. Marchok, and Mary Lynn Baeck. Modeling extreme rainfall, winds, and surge from hurricane isabel (2003). Weather and Forecasting, 25(5):1342–1361, 2010. doi: 10.1175/2010WAF2222349.1. URL https://doi.org/10.1175/2010WAF2222349.1.

Mark Loveland, Amin Kiaghadi, Clint N. Dawson, Hanadi S. Rifai, Shubhra Misra, Helena Mosser, and Alessandro Parola. Developing a modeling framework to simulate compound flooding: When storm surge interacts with riverine flow. Frontiers in Climate, 2:609610, 2021. doi: 10.3389/fclim.2020.609610. URL https://doi.org/10.3389/fclim.2020.609610.

Lu Lu, Pengzhan Jin, Guofei Pang, Zhongqiang Zhang, and George Em Karniadakis. Learning nonlinear operators via DeepONet based on the universal approximation theorem of operators. Nature Machine Intelligence, 3:218–229, 2021. doi: 10.1038/s42256-021-00302-5. URL https://doi.org/10.1038/s42256-021-00302-5.

Lu Lu, Xuhui Meng, Shengze Cai, Zhiping Mao, Somdatta Goswami, Zhongqiang Zhang, and George Em Karniadakis. A comprehensive and fair comparison of two neural operators (with practical extensions) based on FAIR data. Computer Methods in Applied Mechanics and Engineering, 393:114778, 2022. doi: 10.1016/j.cma.2022.114778. URL https://doi.org/10.1016/j.cma.2022.114778.

Richard A. Luettich, Jr., Joannes J. Westerink, and Norman W. Scheffner. ADCIRC: An advanced three-dimensional circulation model for shelves, coasts, and estuaries. report 1: Theory and methodology of ADCIRC-2DDI and ADCIRC-3DL. Dredging Research Program Technical Report DRP-92-6, U.S. Army Engineer Waterways Experiment Station, Vicksburg, Mississippi, November 1992. URL https://adcirc.org/wp-content/uploads/sites/2255/2018/11/1992_Luettich02.pdf.

Mario Morales-Hernández, Pilar García-Navarro, Javier Burguete, and Pilar Brufau. A conservative strategy to couple 1D and 2D models for shallow water flow simulation. Computers & Fluids, 81:26–44, 2013. doi: 10.1016/j.compfluid.2013.04.001. URL https://doi.org/10.1016/j.compfluid.2013.04.001.

NOAA National Centers for Environmental Information. Billion-dollar weather and climate disasters. https://www.ncei.noaa.gov/access/billions/summary-stats, 2025.

Benjamin Pachev, Prateek Arora, Carlos del Castillo-Negrete, Eirik Valseth, and Clint Dawson. A framework for flexible peak storm surge prediction. Coastal Engineering, 186:104406, 2023a. doi: 10.1016/j.coastaleng.2023.104406. URL https://doi.org/10.1016/j.coastaleng.2023.104406.

Benjamin Pachev, L. Ruby Leung, Tian Zhou, and Clint Dawson. One-way coupling of E3SM with ADCIRC demonstrated on hurricane harvey. Natural Hazards, 119(3):2063–2087, 2023b. doi: 10.1007/s11069-023-06192-7. URL https://doi.org/10.1007/s11069-023-06192-7.

Maziar Raissi, Paris Perdikaris, and George E. Karniadakis. Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. Journal of Computational Physics, 378:686–707, 2019. doi: 10.1016/j.jcp.2018.10.045.

Shoshana Reich, Edward Buskey, Clint Dawson, and Eirik Valseth. Potential effects of loading terminal locations on surface trajectories of oil spill transport, 2025. URL https://arxiv.org/abs/2505.24610.

Brad J. Reinhart and Amanda Reinhart. Hurricane otis (EP182023), 22–25 october 2023. Tropical Cyclone Report EP182023, National Hurricane Center, March 2024. URL https://www.nhc.noaa.gov/data/tcr/EP182023_Otis.pdf.

Nicoletta Sanvitale and Elisabeth T. Bowman. Using PIV to measure granular temperature in saturated unsteady polydisperse granular flows. Granular Matter, 18(3):57, 2016. doi: 10.1007/s10035-016-0620-6. URL https://doi.org/10.1007/s10035-016-0620-6.

Danyi Sun, Wenyu Huang, Yong Luo, Jingjia Luo, Jonathon S. Wright, Haohuan Fu, and Bin Wang. A deep learning-based bias correction method for predicting ocean surface waves in the northwest pacific ocean. Geophysical Research Letters, 49(23):e2022GL100916, 2022. doi: 10.1029/2022GL100916. URL https://doi.org/10.1029/2022GL100916.

Timothy Tiggeloven, Anaïs Couasnon, Chiem van Straaten, Sanne Muis, and Philip J. Ward. Exploring deep learning capabilities for surge predictions in coastal areas. Scientific Reports, 11:17224, 2021. doi: 10.1038/s41598-021-96674-0. URL https://doi.org/10.1038/s41598-021-96674-0.

Fang Wang and Di Tian. On deep learning-based bias correction and downscaling of multiple climate model simulations. Climate Dynamics, 59:3451–3468, 2022. doi: 10.1007/s00382-022-06277-2. URL https://doi.org/10.1007/s00382-022-06277-2.

Peitao Wang, Zhiyuan Ren, Lining Sun, Jingming Hou, Zongchen Wang, Ye Yuan, and Fujiang Yu. Observations and modelling of the travel time delay and leading negative phase of the 16 september 2015 illapel, chile tsunami. Acta Oceanologica Sinica, 40:11–30, 2021a. doi: 10.1007/s13131-021-1830-2. URL https://doi.org/10.1007/s13131-021-1830-2.

Sifan Wang, Hanwen Wang, and Paris Perdikaris. Learning the solution operator of parametric partial differential equations with physics-informed DeepONets. Science Advances, 7 (40):eabi8605, 2021b. doi: 10.1126/sciadv.abi8605. URL https://doi.org/10.1126/sciadv.abi8605.

Joannes J. Westerink, Richard A. Luettich, Jr., António M. Baptista, Norman W. Scheffner, and P. Farrar. Tide and storm surge predictions using a finite element model. Journal of Hydraulic Engineering, 118(10):1373–1390, 1992. doi: 10.1061/(ASCE)0733-9429(1992)118:10(1373). URL https://doi.org/10.1061/(ASCE)0733-9429(1992)118:10(1373).

Joannes J. Westerink, Cheryl A. Blain, Richard A. Luettich, Jr., and Norman W. Scheffner. ADCIRC: An advanced three-dimensional circulation model for shelves, coasts, and estuaries. report 2: User's manual for ADCIRC-2DDI. Dredging Research Program Technical Report DRP-92-6, U.S. Army Engineer Waterways Experiment Station, Vicksburg, Mississippi, January 1994. URL https://adcirc.org/wp-content/uploads/sites/2255/2018/11/1994_Westerink01.pdf.

Joannes J. Westerink, Richard A. Luettich, Jesse C. Feyen, John H. Atkinson, Clint Dawson, Hugh J. Roberts, Mark D. Powell, Jason P. Dunion, Ethan J. Kubatko, and Hasan Pourtaheri. A basin- to channel-scale unstructured grid hurricane storm surge model applied to southern louisiana. Monthly Weather Review, 136(3):833–864, 2008. doi: 10.1175/2007MWR1946.1. URL https://doi.org/10.1175/2007MWR1946.1.

Yoshiki Yamazaki, Kwok Fai Cheung, and Zygmunt Kowalik. Depth-integrated, non-hydrostatic model with grid nesting for tsunami generation, propagation, and run-up. International Journal for Numerical Methods in Fluids, 67(12):2081–2107, 2011. doi: 10.1002/fld.2485. URL https://doi.org/10.1002/fld.2485.

Liu Yang, Xuhui Meng, and George Em Karniadakis. B-PINNs: Bayesian physics-informed neural networks for forward and inverse PDE problems with noisy data. Journal of Computational Physics, 425:109913, 2021. doi: 10.1016/j.jcp.2020.109913.